ทักษะปัญญาประดิษฐ์กับวิศวกรรมข้อมูลคลาวด์

BigQuery และการเรียนรู้ของเครื่อง

Series of AI SKILL — Volume 05: AI for Cloud Data Engineering, BigQuery & Machine Learning

ผู้ช่วยศาสตราจารย์ ดร.ชีวะ ทัศนา

สาขาวิชาฟิสิกส์ คณะวิทยาศาสตร์และเทคโนโลยี มหาวิทยาลัยราชภัฏรำไพพรรณี

ศูนย์ความเป็นเลิศด้านสิ่งแวดล้อมและพลังงาน (LEQs) • พ.ศ. 2569

Contents

บทที่ 1

การออกแบบและปรับแต่งประสิทธิภาพ SQL บน Google BigQuery

Google BigQuery เป็นระบบคลังข้อมูลระดับองค์กรที่ใช้สถาปัตยกรรมแยกส่วนการคำนวณออกจากพื้นที่จัดเก็บข้อมูล (Decoupled Compute and Storage) ทำให้สามารถขยายขีดความสามารถในการสืบค้นข้อมูลขนาดมหาศาลได้อย่างรวดเร็ว

1.1 สถาปัตยกรรมกระจายการประมวลผล Dremel และ Colossus

ระบบ Dremel จัดแบ่งงานคิวรีออกเป็นโครงสร้างต้นไม้ (Execution Tree) โดยมี Slot เป็นหน่วยประมวลผลเสมือน การลดปริมาณข้อมูลที่อ่านจากดิสก์ Colossus จึงเป็นปัจจัยชี้ขาดต่อทั้งความเร็วและค่าใช้จ่าย

1.2 เทคนิค Partitioning และ Clustering

การแบ่งพาร์ติชัน (Partitioning) ตามคอลัมน์วันที่หรือเวลา ช่วยให้คำสั่งคิวรีทำการ Pruning ข้อมูลเฉพาะช่วงที่ต้องการ:

$$\text{Data Scanned Ratio} = \frac{\sum \text{Bytes of Filtered Partitions}}{\sum \text{Bytes of Entire Table}} \ll 1 \tag{1.1}$$

เมื่อใช้ร่วมกับการจัดกลุ่มข้อมูล (Clustering) ตามคอลัมน์ที่มีการกรองบ่อยครั้ง เช่น รหัสสถานีตรวจวัด หรือรหัสประเภทเซนเซอร์ BigQuery จะสามารถข้ามการอ่านบล็อกข้อมูลที่ไม่เกี่ยวข้องได้มากกว่าร้อยละ 80

1.3 กลยุทธ์การเขียนคำสั่ง SQL เพื่อหลีกเลี่ยง Data Shuffling

การเชื่อมโยงตารางขนาดใหญ่ (Large Joins) ควรกำหนดให้ตารางที่มีขนาดเล็กกว่าอยู่ทางฝั่งขวา เพื่อให้ระบบทำการบรอดแคสต์ (Broadcast Hash Join) แทนที่จะกระจายข้อมูลข้ามโหนด ซึ่งช่วยลดเวลาการประมวลผลและป้องกันข้อผิดพลาดจากทรัพยากร Slot ไม่เพียงพอ

บทที่ 2

การสร้างโมเดิร์นไปป์ไลน์ ELT ด้วย Dataform และ dbt

การเปลี่ยนกระบวนทัศน์จาก ETL แบบดั้งเดิมสู่ ELT (Extract, Load, Transform) ทำให้การแปลงรูปข้อมูลเกิดขึ้นภายในคลังข้อมูลโดยตรง ซึ่งช่วยเพิ่มความเร็วและลดความซับซ้อนของโครงสร้างพื้นฐาน

2.1 สถาปัตยกรรมเหรียญทองสำหรับคลังข้อมูลวิจัย

ข้อมูลที่ไหลเข้าสู่ระบบถูกจัดระดับความบริสุทธิ์ออกเป็น 3 ชั้น (Medallion Layers):

1. **Bronze Layer (Raw Data):** ข้อมูลดิบจากเซนเซอร์ภาคสนามที่นำเข้าโดยตรง ไม่มีการแปลงสภาพ
2. **Silver Layer (Cleaned & Conformed):** ข้อมูลที่ผ่านการขจัดค่าสูญหาย การปรับสเกล และการตรวจสอบชนิดข้อมูล
3. **Gold Layer (Aggregated Business Level):** ตารางสรุปค่าเฉลี่ยสถิติ ดัชนีสุขภาพดิน หรือค่าชี้วัดที่พร้อมสำหรับนำไปวิเคราะห์และสร้างแดชบอร์ด

2.2 การสร้าง Dataform Actions ด้วย SQLX

Dataform ใช้ไฟล์นามสกุล `.sqlx` ซึ่งผสมผสานระหว่างคำสั่ง SQL และ JavaScript ทำให้สามารถกำหนดตรรกะแบบไดนามิกและการประกาศความสัมพันธ์แบบ Directed Acyclic Graph (DAG) ได้อย่างเป็นระบบ

[กิจกรรมปฏิบัติการ] การเขียน Assertion เพื่อตรวจสอบคุณภาพข้อมูล

การกำหนดเงื่อนไข `assertions` ในไฟล์ SQLX ช่วยให้ระบบหยุดการประมวลผลทันทีหากพบว่าค่าความเป็นกรด-ด่างของดินอยู่นอกช่วงสมเหตุสมผลทางฟิสิกส์ ($0 \leq \text{pH} \leq 14$)

บทที่ 3

การประมวลผล Big Data และโมเดลการเรียนรู้ของเครื่องด้วย Dataproc Serverless

สำหรับงานประมวลผลข้อมูลขนาดใหญ่ที่ไม่เหมาะกับโครงสร้างตารางเชิงสัมพันธ์ Apache Spark บน Dataproc Serverless มอบพลังการคำนวณแบบกระจายศูนย์โดยไม่ต้องเสียเวลาจัดการคลัสเตอร์เสมือน

3.1 สถาปัตยกรรม Apache Spark Serverless

ผู้ใช้งานเพียงส่งไฟล์สคริปต์ PySpark และกำหนดคุณลักษณะของทรัพยากร ระบบของ Google Cloud จะทำการจัดสรรหน่วยคำนวณและปิดระบบลงอัตโนมัติเมื่อประมวลผลเสร็จสิ้น (Just-in-Time Provisioning) ทำให้คิดค่าใช้จ่ายตามระยะเวลาการทำงานจริงระดับวินาที

3.2 การบริหารหน่วยความจำเพื่อป้องกันข้อผิดพลาด Out-Of-Memory

การจัดสรรหน่วยความจำของ Spark Executor ต้องคำนึงถึงสัดส่วนระหว่างพื้นที่จัดเก็บ (Storage Memory) และพื้นที่คำนวณ (Execu-

tion Memory):

$$\text{Total Executor Memory} = \text{Overhead} + \text{Off-Heap} + \text{On-Heap}(\text{Storage} + \text{Execu} \tag{3.1}$$

การใช้คำสั่ง `repartition()` อย่างเหมาะสม ร่วมกับการหลีกเลี่ยงคำสั่ง `collect()` บนข้อมูลขนาดใหญ่ ช่วยให้การคำนวณไหลลื่นโดยไม่เกิดอาการ Driver Crash

3.3 การสร้างไปป์ไลน์ Machine Learning ด้วย Spark MLlib

การฝึกสอนโมเดลทำนาย เช่น การพยากรณ์ความชื้นในดินล่วงหน้าด้วย Random Forest Regressor สามารถรันบนคลัสเตอร์หลายร้อยคอร์พร้อมกัน ทำให้ลดเวลาการฝึกสอนโมเดลจากหลายชั่วโมงเหลือเพียงไม่กี่นาที